POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

IoT Homework 2024/2025
PART 1 – Exercise 1

Tesi di Laurea Magistrale in
Telecommunication Engineering

Author: **Hong CHEN** [redacted]
Anan WANG [redacted]

Student ID: [redacted] & [redacted]
Advisor:
Co-advisors:

Academic Year: 2025-05

Contents

1 | Exercise 1

1.1. Objective

Design a low-cost and scalable IoT system to achieve the following:

- **Real-Time Localization:** Covering a 500 m^2 underground warehouse and a 1 km^2 outdoor yard.
- **Status Monitoring:** Daily distance traveled, max/avg speed, and impact detection.

1.2. System Architecture Overview

This project designs a forklift tracking and status monitoring system for warehouse environments, including GPS-denied underground areas. The system focuses on low power, low cost, and scalability. Each forklift is equipped with sensors (e.g., IMU, GPS, BLE, collision detector) and an ESP32 controller that sends data via LoRa to an in-warehouse gateway. The data is then uploaded to the cloud using MQTT for real-time processing, storage, and visualization.
The proposed system consists of three layers:

- **Edge Devices:** Forklift-mounted devices collecting real-time data such as location, speed, and impact.
- **Communication Layer:** Data is transmitted via LoRa to the cloud.
- **Backend Platform:** Platform for receiving, processing, storing, and visualizing the data

1.2.1. Forklift-mounted Edge Devices

The forklift edge devices collect key data such as position, speed, and collisions, and transmit them in real time via low-power communication modules. Related sensors and protocols are listed in Table 1.1.

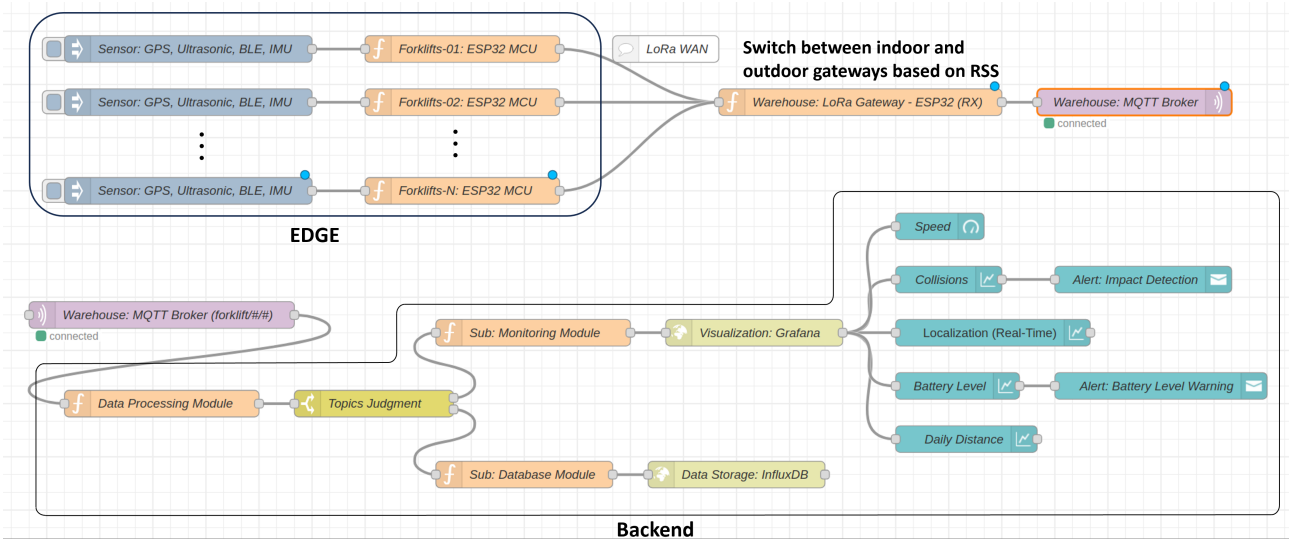

Figure 1.1: Exercise 1: Module Setup

Edge Layer: Forklift Hardware	
Localization Module	Outdoor: GPS+IMU
	Indoor: BLE Beacon + IMU
Status Monitoring	IMU: Detects acceleration, angular motion, and collision events.
	Ultrasonic Sensors: Collision avoidance and obstacle detection
	Speed: IMU + Wheel Encoder
Processing and Communication Unit	MCU (ESP32): Controls sensing logic, data packaging, and communication.
	LoRa: Transmits data over long-range LoRa network.

Table 1.1: Forklift Edge Layer Hardware Components

1.2.2. Communication Layer: Network Design

To ensure reliable, low-cost, and scalable communication between mobile forklifts and the cloud platform across both indoor (basement) and outdoor (ground) areas, our system adopts a LoRa + MQTT-based architecture, combining long-range, low-power wireless transmission with an efficient backend message routing system.

To guarantee reliable coverage across the entire warehouse area, we deploy three LoRa gateways:

- Two gateways installed on the rooftop or high points of the warehouse cover the 1 km^2 outdoor area, while forklifts automatically switch between gateways based on RSSI values to determine whether they are on the ground floor or in the basement.
- One gateway in the center of the basement covers the 500 m^2 underground area.

```
{
  "forklift_id": "F001",
  "timestamp": "2025-05-16T10:23:00Z",
  "location": {"lat": 45.478, "lon": 9.232, "floor": "underground"},
  "speed": 3.2,
  "acceleration": [0.1, 0.0, -0.3],
  "collision": false,
  "battery": 27
}
```

Figure 1.2: Data Flows (JSON format)

These gateways operate in parallel and do not interfere with each other due to physical separation and backend redundancy.
Each forklift is equipped with a single LoRa module. It does not need to know which gateway is currently connected. All gateways listening on the same frequency will receive the signal.

- When a forklift transmits a packet, multiple gateways may receive it.
- Gateways forward the packet to a central MQTT broker in the warehouse backend.
- LoRa Gateway: Receives LoRa data and forwards them via Wi-Fi / Ethernet.
- The backend server duplicates and selects the best signal using RSSI/SNR.
- As forklifts move across zones, this process automatically ensures seamless roaming.

Each forklift sends its data every 10 seconds to a LoRa gateway. The gateway forwards the data to a warehouse MQTT Broker using the MQTT protocol.
The Broker routes messages to specific topics, where backend components are subscribed.

1.2.3. Backend Platform

The cloud platform receives MQTT data sent from forklifts via LoRa gateways, performs data processing, storage, and visualization. Mosquitto acts as a high-performance MQTT Broker, Node-RED handles data routing and alerts, InfluxDB stores time-series data, and Grafana visualizes real-time location, battery status, and alerts, ensuring an efficient and stable system.

- MQTT Broker (Mosquitto): Receives data from forklifts and distributes it to subscribers.
- Data Processing (Node-RED): Parses data, triggers alerts (e.g., collision, low battery), forwards data for storage and visualization.
- Data Storage (InfluxDB): Stores time-series data such as forklift trajectories, speed, and battery levels.
- Visualization (Grafana): Displays real-time maps, status panels, and alert logs.

- Alert Module: Automatically sends notifications based on rules to ensure safety and maintenance.

The pseudocode below outlines the core workflow of the forklift device, including sensor data collection (GPS, BLE, IMU), battery monitoring, collision detection, and packaging the collected data into a unified JSON format sent via LoRa to the gateway.

Listing 1 Forklift Tracking System Pseudocode

```
INIT:
    Initialize sensors: GPS, BLE, IMU, Ultrasonic, Wheel Encoder
    Initialize LoRa module and MQTT settings
    Set timer to trigger every 10 seconds
    Define thresholds for battery and collision

FUNCTION build_json_packet(event_type):
    location = get_location()  # GPS if available, else BLE
    speed = read_wheel_encoder()
    battery = read_battery()
    imu_data = read_IMU()

    packet = {
        "id": forklift_id,
        "timestamp": current_time,
        "type": event_type,
        "location": location,
        "speed": speed,
        "acceleration": imu_data,
        "collision": (event_type == "collision_alert"),
        "battery": battery
    }

    return packet

LOOP (runs continuously):

    # 1. Collision monitoring (runs continuously)
    imu_data = read_IMU()
    ultrasonic_data = read_ultrasonic()
    collision_flag = detect_collision(imu_data, ultrasonic_data)

    IF collision_flag == TRUE:
        trigger_brake()
        trigger_buzzer()

        alert_packet = build_json_packet("collision_alert")
        send_packet_via_LoRa(alert_packet)

    # 2. Periodic data collection (every 10 seconds)
    IF timer_triggered():
        battery = read_battery()

        IF battery < threshold:
            set_destination_to_charging_station()

        telemetry_packet = build_json_packet("telemetry")
        send_packet_via_LoRa(telemetry_packet)

```